

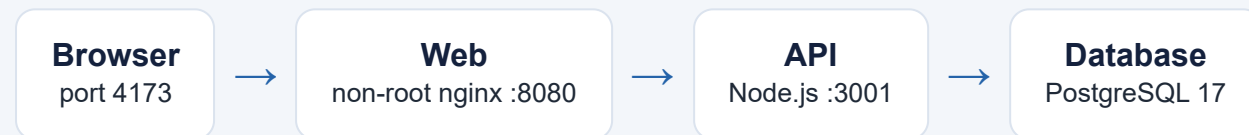
ARCHITECTURE OVERVIEW

Full-Stack CI/CD Pipeline

React + Node.js API + PostgreSQL, released as immutable containers, with a live Pipeline Monitor that visualizes GitHub Actions and staging health.

APPLICATION

Runtime architecture



Single public origin

nginx serves React and proxies `/api/*`. Only port 4173 is published.

Network split

PostgreSQL stays on an internal network. The API also joins an egress network so it can call the GitHub API for live status.

Reusable workflow layout

```
ci.yml cd.yml
├ reusable-test.yml │ reusable-deploy.yml
├ reusable-security.yml └ reusable-rollback.yml
└ reusable-docker.yml
```

Reusable workflows sit flat under `.github/workflows`. CI validates; CD deploys digests over SSH.

CONTINUOUS INTEGRATION

Test the complete system

Web

- ESLint + Prettier
- TypeScript
- Vitest coverage

API + database

- API unit tests
- PostgreSQL service
- Migration + integration test

Full-stack E2E

- Docker Compose
- Real contact API
- Five Playwright browsers

SECURITY

Gates across both applications

Dependencies

Separate web and API lockfile audits; reviewed dependency upgrades only.

Source

TruffleHog, CodeQL, and Trivy filesystem SARIF.

Containers

SBOMs, provenance, blocking HIGH/CRITICAL Trivy scans, and pinned OS packages (e.g. `libexpat`) with cache-busting rebuilds.

RELEASE

Build once, publish by CalVer

```
ghcr.io/<owner>/platform-web:YYYY.MM.N
```

```
ghcr.io/<owner>/platform-api:YYYY.MM.N
```

Traceable

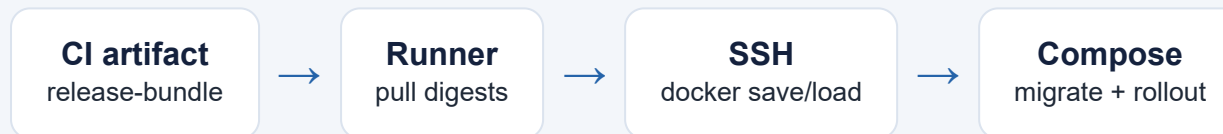
:staging, :sha-short, SBOM, git revision, and provenance.

Immutable deployment

CD deploys image@sha256:digest, never a mutable tag. Images are transferred to the VM over SSH.

CONTINUOUS DELIVERY

Staging deployment



The VM does not need GHCR access. Tailscale is optional; host keys remain pinned. Deploy overlays the latest `compose.yml` and deploy scripts.

LIVE MONITORING

Pipeline Monitor UI

Overview

Latest CI/CD runs, stage flow, and plain-language status.

Runs

Filter CI vs CD, open a run for jobs and stages.

Artifacts

Coverage, Playwright reports, release bundles.

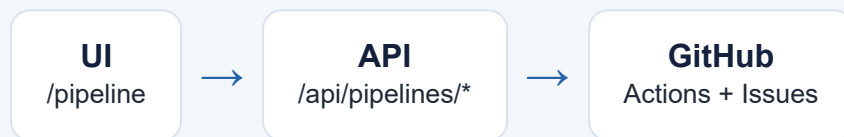
Failures

Open ci-failure issues plus staging smoke probes.

Route: /pipeline. Light UI, auto-refresh while a run is in progress.

LIVE MONITORING

API BFF for GitHub Actions



Token stays server-side

Local: `GITHUB_TOKEN`, `OWNER`, `REPO` in `.env`. Staging:
`PIPELINE_GITHUB_TOKEN` written into the API env by CD.

Safe when unconfigured

`/status` always answers. Other pipeline routes return 503 with a setup hint; the UI shows a banner and skips those fetches.

STAGING NETWORK

Why the API needs egress

backend (internal)

Web ↔ API ↔ PostgreSQL. No outbound internet. Keeps the database private.

egress

API-only path for HTTPS to `api.github.com`. Without it, overview fails on the VM even when the token is set.

CD smoke-checks `/api/pipelines/status` (configured: `true`) and `/api/pipelines/overview`.

RESILIENCE

Health and rollback

Release validation

- PostgreSQL health
- API readiness
- web health
- external smoke routes
- pipeline status + overview

Failure path

- restore prior web/API digests
- preserve PostgreSQL volume
- do not reverse migrations
- create deployment issue

FEEDBACK

Issues, wiki, and artifacts

Failure issues

Full-stack tests, security, container release, migration, Compose deploy, and rollback.

Wiki reports

Run metadata, stage results, job links, remediation, and release artifacts.

Artifacts

web-dist, coverage, Playwright report, release metadata, Compose, scripts, SBOMs.

SUMMARY

Build. Ship. Watch.

PR validation → secure immutable release → Compose staging → automatic rollback → live Pipeline
Monitor on `/pipeline`.